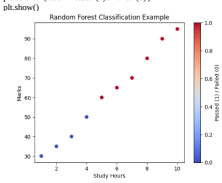
7 Implement random forest classifier

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Step 1: Create a simple dataset for classification
data = {
    'Study_Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],  # Number of hours studied
    'Marks': [30, 35, 40, 50, 60, 65, 70, 80, 90, 95],  # Marks obtained
    'Passed': [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]  # 0=Fail, 1=Pass
}

# Convert the data into a pandas DataFrame
df = pd.DataFrame(data)

# Step 2: Visualize the data
plt.scatter(df['Study_Hours'], df['Marks'], c=df['Passed'], cmap='coolwarm', marker='o')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.title('Random Forest Classification Example')
plt.colorbar(label='Passed (1) / Failed (0)')
plt.show()

# Step 3: Split the data into features (X) and target (y)
X = df[['Study_Hours', 'Marks']]  # Features (independent variables)
y = df['Passed']  # Target (dependent variable)

# Step 4: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 5: Initialize and train the Random Forest Classifier model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 6: Make predictions using the trained model
y_pred = model.predict(X_test)

# Step 7: Evaluate the model performance
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

# Output the evaluation results
print(f'Accuracy: {accuracy}')
```

```
print(f'Confusion Matrix:\n{conf_matrix}')
print(f'Classification Report:\n{class_report}')

# Step 8: Visualize the Random Forest predictions
# Plot the original data points
plt.scatter(df['Study_Hours'], df['Marks'], c=df['Passed'], cmap='coolwarm', marker='o')

# Plot the predictions from the Random Forest model
# Here, we will use the predicted probabilities of passing (class 1)
plt.scatter(X_test['Study_Hours'], X_test['Marks'], c=y_pred, marker='x', s=100,
label='Predictions', edgecolor='black')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.title('Random Forest Classifier - Predictions vs Actual')
plt.legend()
plt.colorbar(label='Passed (1) / Failed (0)')
plt.show()
```




```
Accuracy: 1.0
Confusion Matrix:
[[1 0]
 [0 2]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

8) Object segmentation using hierarchical based methods

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, color
from skimage.segmentation import slic, mark_boundaries
from sklearn.cluster import AgglomerativeClustering
# Step 1: Load the image
image = io.imread('sample.jpg')
image = image / 255.0            # Normalize
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')
plt.show()
# Step 2: Generate Superpixels
segments = slic(image, n_segments=200, compactness=10,
start_label=0)
plt.imshow(mark_boundaries(image, segments))
plt.title("Superpixels")
plt.axis('off')
plt.show()
# Step 3: Extract features of each segment
num_segments = np.max(segments) + 1
features = []
for i in range(num_segments):
    mask = segments == i
    mean_color = image[mask].mean(axis=0)
    features.append(mean_color)
features = np.array(features)
# Step 4: Hierarchical Clustering
n_clusters = 5   # number of final regions
clustering = AgglomerativeClustering(n_clusters=n_clusters)
labels = clustering.fit_predict(features)
# Step 5: Map cluster labels back to image
segmented_image = np.zeros_like(segments)
for i in range(num_segments):
    segmented_image[segments == i] = labels[i]
# Step 6: Display segmented result
plt.imshow(segmented_image, cmap='viridis')
plt.title("Hierarchical Segmented Image")
plt.axis('off')
plt.show()
```

9) perform visualition techniques like bar,column ,line,scatter,3d cubes

```
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# Sample data
x = np.array([1, 2, 3, 4, 5])
y = np.array([10, 25, 15, 30, 20])

# ----------------------------
# Bar Chart
# ----------------------------
plt.figure()
plt.bar(x, y)
plt.title("Bar Chart")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()

# ----------------------------
# Column Chart
# ----------------------------
plt.figure()
plt.barh(x, y, color='skyblue')   # horizontal bars
plt.title("Column chart)")
plt.xlabel("Values")
plt.ylabel("Categories")
plt.show()

# ----------------------------
# Line Chart
# ----------------------------
plt.figure()
plt.plot(x, y, marker='o')
plt.title("Line Chart")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
```

```
# ----------------------------
# Scatter Plot
# ----------------------------
plt.figure()
plt.scatter(x, y, color='red')
plt.title("Scatter Plot")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()

# ----------------------------
# 3D Cubes (3D Bar Blocks)
# ----------------------------
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

xpos = np.arange(len(x))
ypos = np.zeros(len(x))
zpos = np.zeros(len(x))

dx = np.ones(len(x))
dy = np.ones(len(x))
dz = y

ax.bar3d(xpos, ypos, zpos, dx, dy, dz)
ax.set_title("3D Cubes")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Height")

plt.show()
```

10) Perform Descriptive analytics on healthcare data

```
# Step 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Step 2: Load the dataset
df = pd.read_csv("heart.csv")
# Step 3: Display first 5 records
print("First 5 Records:")
print(df.head())
# Step 4: Basic Information
print("\nShape of Dataset:", df.shape)
print("\nColumn Names:\n", df.columns)
print("\nData Types:\n", df.dtypes)
print("\nMissing Values:\n", df.isnull().sum())
# Step 5: Descriptive Statistics
print("\nStatistical Summary:")
print(df.describe())
# Step 6: Age Distribution
plt.figure()
sns.histplot(df['age'], kde=True)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
# Step 7: Gender Distribution
plt.figure()
sns.countplot(x='sex', data=df)
plt.title("Gender Distribution (0=Female, 1=Male)")
plt.show()
# Step 8: Heart Disease Count
plt.figure()
sns.countplot(x='target', data=df)
plt.title("Heart Disease Count (0=No Disease, 1=Disease)")
plt.show()
# Step 9: Cholesterol Boxplot
plt.figure()
sns.boxplot(x=df['chol'])
plt.title("Cholesterol Boxplot")
plt.show()
# Step 10: Correlation Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

11) Perform Predictive analytics on Product Sales data .

```
# Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
# Step 2: Load Dataset
data = pd.read_csv("train.csv")
print(data.head())
# Step 3: Convert Date Column
data['Order Date'] = pd.to_datetime(data['Order Date'], dayfirst=True)
# Extract Month
data['Month'] = data['Order Date'].dt.month
# Step 4: Convert Categorical Columns into Numbers
data = pd.get_dummies(data, columns=['Category','Region'], drop_first=True)
# Step 5: Select Features
X = data[['Month','Category_Office Supplies','Category_Technology',
          'Region_South','Region_West','Region_East']]
y = data['Sales']
# Step 6: Split Dataset
X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=42)
# Step 7: Create Model
model = LinearRegression()
# Step 8: Train Model
model.fit(X_train,y_train)
# Step 9: Predict Sales
y_pred = model.predict(X_test)
print("Predicted Sales:")
print(y_pred[:10])
# Step 10: Evaluate Model
print("MAE:",mean_absolute_error(y_test,y_pred))
print("R2 Score:",r2_score(y_test,y_pred))
# Step 11: Visualization
plt.figure(figsize=(8,5))
plt.plot(y_test.values[:50], label="Actual Sales", marker='o')
plt.plot(y_pred[:50], label="Predicted Sales", marker='x')
plt.xlabel("Data Points")
plt.ylabel("Sales")
plt.title("Actual vs Predicted Sales Comparison")
plt.legend()
plt.show()
```

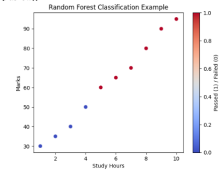
12) Apply Predictive analytics for Weather forecasting.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load dataset
data = pd.read_csv("weather.csv")

# Features and target
X = data[['Humidity','Wind Speed (km/h)','Pressure (millibars)','Visibility
(km)']]
y = data['Temperature (C)']

# Train-test split
X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train,y_train)

# Prediction
y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# Plot graph
plt.figure()

plt.plot(range(50), y_test[:50], label="Actual Temperature")

plt.plot(range(50), y_pred[:50], label="Predicted Temperature")

plt.xlabel("Data Points")
plt.ylabel("Temperature")
plt.title("Actual vs Predicted Temperature")
plt.legend()

plt.show()
```


7 Implement random forest classifier

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Step 1: Create a simple dataset for classification
data = {
    'Study_Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],  # Number of hours studied
    'Marks': [30, 35, 40, 50, 60, 65, 70, 80, 90, 95],  # Marks obtained
    'Passed': [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]  # 0=Fail, 1=Pass
}

# Convert the data into a pandas DataFrame
df = pd.DataFrame(data)

# Step 2: Visualize the data
plt.scatter(df['Study_Hours'], df['Marks'], c=df['Passed'], cmap='coolwarm', marker='o')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.title('Random Forest Classification Example')
plt.colorbar(label='Passed (1) / Failed (0)')
plt.show()

# Step 3: Split the data into features (X) and target (y)
X = df[['Study_Hours', 'Marks']]  # Features (independent variables)
y = df['Passed']  # Target (dependent variable)

# Step 4: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 5: Initialize and train the Random Forest Classifier model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 6: Make predictions using the trained model
y_pred = model.predict(X_test)

# Step 7: Evaluate the model performance
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

# Output the evaluation results
print(f'Accuracy: {accuracy}')
```

```
print(f'Confusion Matrix:\n{conf_matrix}')
print(f'Classification Report:\n{class_report}')

# Step 8: Visualize the Random Forest predictions
# Plot the original data points
plt.scatter(df['Study_Hours'], df['Marks'], c=df['Passed'], cmap='coolwarm', marker='o')

# Plot the predictions from the Random Forest model
# Here, we will use the predicted probabilities of passing (class 1)
plt.scatter(X_test['Study_Hours'], X_test['Marks'], c=y_pred, marker='x', s=100,
label='Predictions', edgecolor='black')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.title('Random Forest Classifier - Predictions vs Actual')
plt.legend()
plt.colorbar(label='Passed (1) / Failed (0)')
plt.show()
```




```
Accuracy: 1.0
Confusion Matrix:
[[1 0]
 [0 2]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

8) Object segmentation using hierarchical based methods

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, color
from skimage.segmentation import slic, mark_boundaries
from sklearn.cluster import AgglomerativeClustering
# Step 1: Load the image
image = io.imread('sample.jpg')
image = image / 255.0            # Normalize
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')
plt.show()
# Step 2: Generate Superpixels
segments = slic(image, n_segments=200, compactness=10,
start_label=0)
plt.imshow(mark_boundaries(image, segments))
plt.title("Superpixels")
plt.axis('off')
plt.show()
# Step 3: Extract features of each segment
num_segments = np.max(segments) + 1
features = []
for i in range(num_segments):
    mask = segments == i
    mean_color = image[mask].mean(axis=0)
    features.append(mean_color)
features = np.array(features)
# Step 4: Hierarchical Clustering
n_clusters = 5   # number of final regions
clustering = AgglomerativeClustering(n_clusters=n_clusters)
labels = clustering.fit_predict(features)
# Step 5: Map cluster labels back to image
segmented_image = np.zeros_like(segments)
for i in range(num_segments):
    segmented_image[segments == i] = labels[i]
# Step 6: Display segmented result
plt.imshow(segmented_image, cmap='viridis')
plt.title("Hierarchical Segmented Image")
plt.axis('off')
plt.show()
```

9) perform visualition techniques like bar,column ,line,scatter,3d cubes

```
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# Sample data
x = np.array([1, 2, 3, 4, 5])
y = np.array([10, 25, 15, 30, 20])

# ----------------------------
# Bar Chart
# ----------------------------
plt.figure()
plt.bar(x, y)
plt.title("Bar Chart")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()

# ----------------------------
# Column Chart
# ----------------------------
plt.figure()
plt.barh(x, y, color='skyblue')   # horizontal bars
plt.title("Column chart)")
plt.xlabel("Values")
plt.ylabel("Categories")
plt.show()

# ----------------------------
# Line Chart
# ----------------------------
plt.figure()
plt.plot(x, y, marker='o')
plt.title("Line Chart")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
```

```
# ----------------------------
# Scatter Plot
# ----------------------------
plt.figure()
plt.scatter(x, y, color='red')
plt.title("Scatter Plot")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()

# ----------------------------
# 3D Cubes (3D Bar Blocks)
# ----------------------------
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

xpos = np.arange(len(x))
ypos = np.zeros(len(x))
zpos = np.zeros(len(x))

dx = np.ones(len(x))
dy = np.ones(len(x))
dz = y

ax.bar3d(xpos, ypos, zpos, dx, dy, dz)
ax.set_title("3D Cubes")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Height")

plt.show()
```

10) Perform Descriptive analytics on healthcare data

```
# Step 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Step 2: Load the dataset
df = pd.read_csv("heart.csv")
# Step 3: Display first 5 records
print("First 5 Records:")
print(df.head())
# Step 4: Basic Information
print("\nShape of Dataset:", df.shape)
print("\nColumn Names:\n", df.columns)
print("\nData Types:\n", df.dtypes)
print("\nMissing Values:\n", df.isnull().sum())
# Step 5: Descriptive Statistics
print("\nStatistical Summary:")
print(df.describe())
# Step 6: Age Distribution
plt.figure()
sns.histplot(df['age'], kde=True)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
# Step 7: Gender Distribution
plt.figure()
sns.countplot(x='sex', data=df)
plt.title("Gender Distribution (0=Female, 1=Male)")
plt.show()
# Step 8: Heart Disease Count
plt.figure()
sns.countplot(x='target', data=df)
plt.title("Heart Disease Count (0=No Disease, 1=Disease)")
plt.show()
# Step 9: Cholesterol Boxplot
plt.figure()
sns.boxplot(x=df['chol'])
plt.title("Cholesterol Boxplot")
plt.show()
# Step 10: Correlation Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

11) Perform Predictive analytics on Product Sales data .

```
# Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
# Step 2: Load Dataset
data = pd.read_csv("train.csv")
print(data.head())
# Step 3: Convert Date Column
data['Order Date'] = pd.to_datetime(data['Order Date'], dayfirst=True)
# Extract Month
data['Month'] = data['Order Date'].dt.month
# Step 4: Convert Categorical Columns into Numbers
data = pd.get_dummies(data, columns=['Category','Region'], drop_first=True)
# Step 5: Select Features
X = data[['Month','Category_Office Supplies','Category_Technology',
          'Region_South','Region_West','Region_East']]
y = data['Sales']
# Step 6: Split Dataset
X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=42)
# Step 7: Create Model
model = LinearRegression()
# Step 8: Train Model
model.fit(X_train,y_train)
# Step 9: Predict Sales
y_pred = model.predict(X_test)
print("Predicted Sales:")
print(y_pred[:10])
# Step 10: Evaluate Model
print("MAE:",mean_absolute_error(y_test,y_pred))
print("R2 Score:",r2_score(y_test,y_pred))
# Step 11: Visualization
plt.figure(figsize=(8,5))
plt.plot(y_test.values[:50], label="Actual Sales", marker='o')
plt.plot(y_pred[:50], label="Predicted Sales", marker='x')
plt.xlabel("Data Points")
plt.ylabel("Sales")
plt.title("Actual vs Predicted Sales Comparison")
plt.legend()
plt.show()
```

12) Apply Predictive analytics for Weather forecasting.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load dataset
data = pd.read_csv("weather.csv")

# Features and target
X = data[['Humidity','Wind Speed (km/h)','Pressure (millibars)','Visibility
(km)']]
y = data['Temperature (C)']

# Train-test split
X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train,y_train)

# Prediction
y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# Plot graph
plt.figure()

plt.plot(range(50), y_test[:50], label="Actual Temperature")

plt.plot(range(50), y_pred[:50], label="Predicted Temperature")

plt.xlabel("Data Points")
plt.ylabel("Temperature")
plt.title("Actual vs Predicted Temperature")
plt.legend()

plt.show()
```